

TNPSC Combined Technical Services

Paper Code 554 — Unit IV

DATABASE MANAGEMENT SYSTEMS

Expanded textbook-style study notes with conceptual diagrams and flow charts

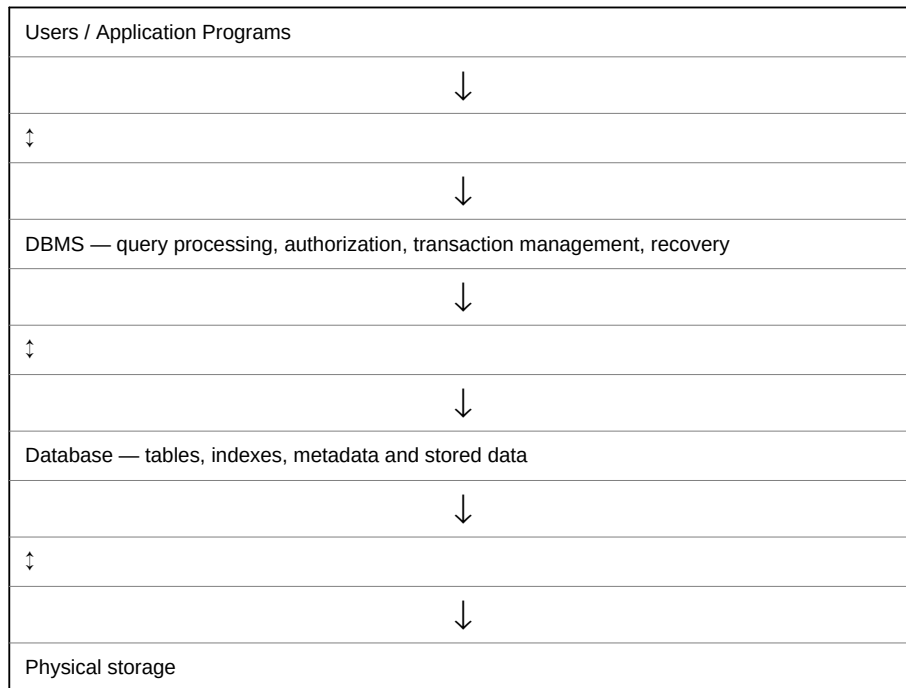
This unit is developed from fundamentals through database design, relational theory, normalization, SQL, query optimization, transactions, concurrency, indexing, hashing and modern database models. Diagrams are included wherever they clarify architecture, process flow or structure.

PART I — INTRODUCTION TO DBMS

Introduction to DBMS

A Database Management System (DBMS) is software that provides an organized way to define, create, store, retrieve, update and control access to data. Instead of allowing each application to manage its own files independently, a DBMS provides a common data-management layer between applications and persistent storage. A database is a collection of related data, while the DBMS is the software responsible for managing that data. A complete database system includes the database, DBMS software, application programs, users and the underlying hardware. The DBMS provides data definition, data manipulation, security, integrity enforcement, concurrency control, transaction management, recovery and efficient query processing. These services are what distinguish a database system from simply storing information in files.

Basic DBMS architecture



File Systems Organization

Before DBMSs became widespread, applications commonly stored data in separate files. A file system can organize records sequentially or through pointers, indexes or direct-address techniques. Although file systems are simple, application programs often become tightly coupled to file formats and storage details. Typical file-system problems include data redundancy, inconsistent copies, difficult ad-hoc querying, weak sharing, limited integrity enforcement, poor concurrency support and recovery complexity. A DBMS addresses these problems by centralizing data management and providing higher-level abstractions.

Sequential File Organization

In sequential organization, records are stored one after another, commonly ordered by a search key. Sequential access is efficient for processing many records in order, such as payroll or batch reports. Searching for an arbitrary record can be expensive because records may need to be scanned until the desired position is reached. Sequential files are particularly suitable when operations are predominantly batch-oriented and records are processed in key order.

Pointer-Based File Organization

Pointer organization connects records using addresses or pointers. A record can contain a pointer to another record, allowing logical relationships between records to be represented without requiring physical adjacency. Pointers can make traversal of related records efficient, but they create maintenance issues when records move or storage is reorganized. Modern DBMSs generally manage physical addresses internally rather than exposing raw storage pointers to ordinary SQL users.

Indexed File Organization

An indexed file maintains a separate index structure that maps search-key values to locations of records. The index allows the system to avoid scanning every record. Indexes are especially valuable for selective queries over large files. An index adds storage and update overhead: when records change, relevant index entries may also need to be updated. Therefore, indexing is a trade-off between faster retrieval and additional maintenance cost.

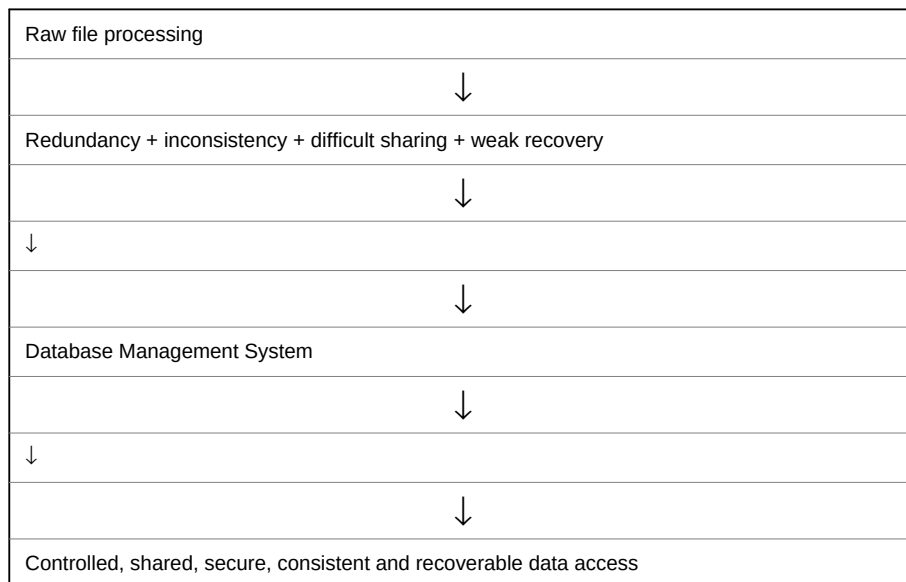
Direct / Hashed File Organization

Direct organization computes a storage location from a key, often through a hash function. When the hash function distributes keys well, records can be located quickly without scanning the file. Collisions occur when multiple keys map to the same location, so a collision-resolution strategy is required.

Purpose of a Database System

A database system exists to make data persistent, shareable, secure, consistent and efficiently accessible. It provides a level of abstraction between applications and physical storage. Major objectives include controlling redundancy, supporting multiple users, enforcing integrity constraints, providing authorization, supporting concurrent transactions, recovering after failures, and allowing users to express queries without specifying low-level file-access procedures.

Why DBMS?



Database System Terminologies

Important terminology includes database, relation/table, tuple/row, attribute/column, domain, schema, instance, primary key, candidate key, foreign key, constraint and metadata. A schema describes the structure of the database and changes relatively infrequently; an instance is the actual contents at a particular time. A relation is mathematically a set of tuples over named attributes, although practical SQL tables may contain duplicate rows unless constrained otherwise. A primary key uniquely identifies rows. A foreign key represents a reference to a candidate/primary key in another relation and helps enforce referential integrity.

Database Characteristics

- Self-describing nature: the system stores metadata describing tables, columns, constraints, indexes and other objects.
- Program-data independence: applications need not know the physical storage representation.
- Data sharing and multi-user access: many users/applications can work with common data under controlled concurrency.
- Controlled redundancy: the design can reduce unnecessary duplication.
- Integrity enforcement: constraints help prevent invalid data.
- Security and authorization: access can be restricted according to users and privileges.

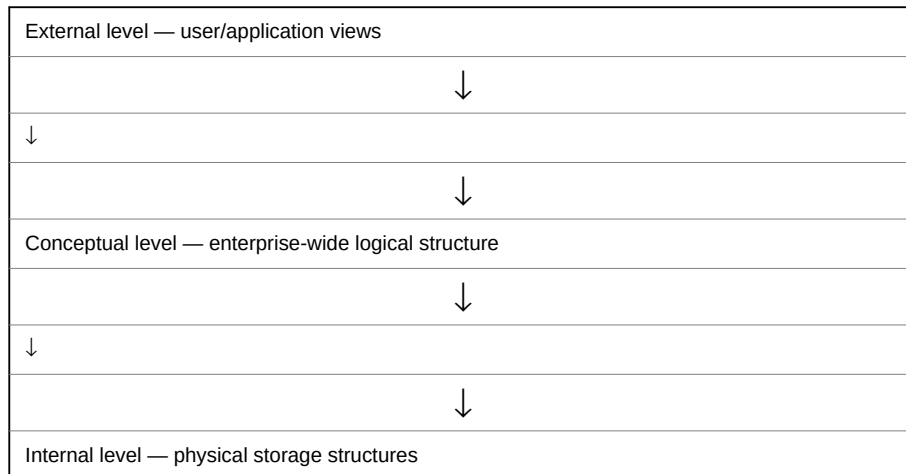
- Backup and recovery: committed data can be protected against failures.
- Transaction support: related operations can be treated as atomic units.

Data Models

A data model is a collection of concepts used to describe data, relationships, constraints and operations. Conceptual models help capture requirements, while logical models organize data in a form suitable for implementation and physical models describe storage details. Major categories include hierarchical, network, relational, entity-relationship, object-oriented and object-relational models. The relational model represents information through relations; the ER model represents entities and relationships at a conceptual level; object-oriented models represent objects with identity, state and behaviour.

Types of Data Models

Three broad levels of data description



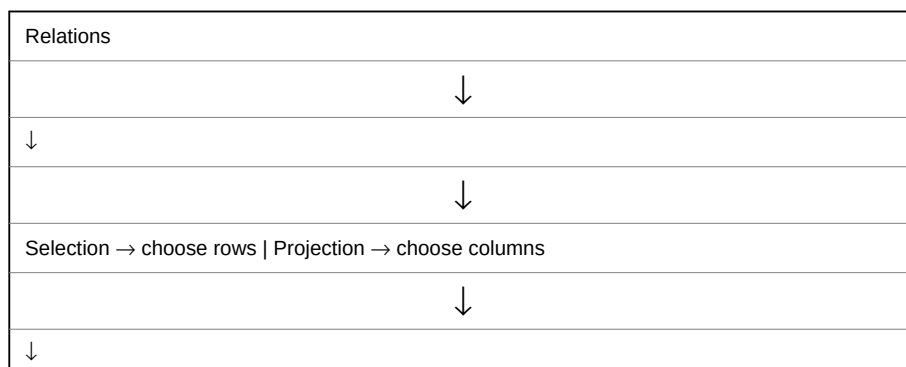
Components of a DBMS

A DBMS typically contains a query processor, storage manager, transaction manager, recovery manager, authorization/integrity subsystem, buffer manager and metadata/catalog facilities. The query processor parses and optimizes queries and generates an execution plan. The storage manager handles files, pages, indexes and access methods. Transaction and recovery components coordinate concurrency and failure handling.

Relational Algebra

Relational algebra is a procedural formal query language for the relational model. It defines operators that take relations as input and produce relations as output. Fundamental operators include selection, projection, union, set difference, Cartesian product and rename. Join operations can be derived or treated as extended relational operators. Selection filters rows according to a predicate. Projection selects attributes/columns. Union combines compatible relations. Difference returns tuples in one relation but not the other. Cartesian product forms combinations of tuples. Joins combine related tuples according to a condition.

Common relational algebra operations



↓
Join → combine related rows Union/Difference → set operations
↓
↓
↓
Result relation

PART II — LOGICAL DATABASE DESIGN

Relational DBMS

A relational DBMS implements the relational model using relations, keys and integrity constraints. Data is represented in tables, while relationships are represented through common attribute values and foreign keys. SQL provides the primary practical interface for defining and manipulating relational data. A relational design separates logical relationships from physical storage. The optimizer and storage engine decide how a declarative query should be executed.

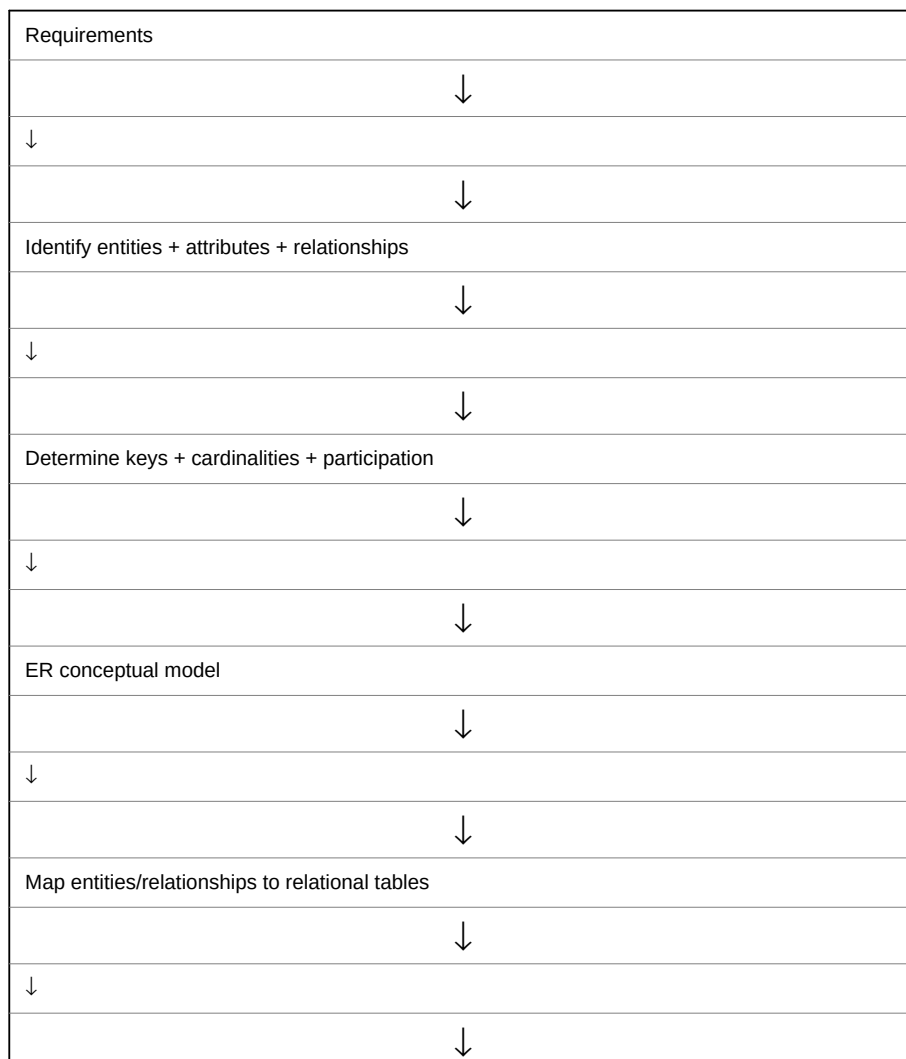
Codd's Rules

E. F. Codd proposed a set of rules describing characteristics expected of a relational DBMS. They emphasize relational representation, logical accessibility, systematic treatment of nulls, catalog accessibility, a comprehensive data sublanguage, view updating where theoretically possible, set-level insert/update/delete, physical and logical data independence, integrity independence, distribution independence and non-subversion. The rules are best understood as a conceptual benchmark for relational systems rather than a checklist where every commercial DBMS behaves identically.

Entity–Relationship (ER) Model

The ER model is a conceptual modeling technique. An entity represents a distinguishable real-world object or concept. An entity set is a collection of similar entities. An attribute describes an entity or relationship. A relationship represents an association among entities. Cardinality specifies how many entities may participate in a relationship, such as one-to-one, one-to-many or many-to-many. Participation constraints describe whether participation is mandatory or optional. A strong ER design separates objects, their properties and their relationships before converting the conceptual model into relational tables.

ER-to-relational design flow



Extended ER Model

The Extended Entity-Relationship (EER) model adds richer concepts such as specialization, generalization, inheritance, categories/union types and aggregation. Specialization divides a superclass into more specific subclasses based on distinguishing characteristics. Generalization combines similar entity types into a higher-level superclass. Inheritance allows a subclass to inherit attributes and relationships from its superclass. EER concepts are useful when the domain contains meaningful class hierarchies that would otherwise be difficult to express clearly in a basic ER diagram.

Functional Dependencies

A functional dependency (FD) $X \rightarrow Y$ means that if two tuples agree on attributes X , they must also agree on attributes Y . X is the determinant and Y is functionally dependent on X . Functional dependencies are central to normalization. A candidate key functionally determines all attributes of a relation. Partial dependencies and transitive dependencies explain why certain decompositions are needed. For example, in $\text{Student}(\text{StudentID}, \text{StudentName}, \text{DepartmentID}, \text{DepartmentName})$, $\text{StudentID} \rightarrow \text{StudentName}, \text{DepartmentID}$ and $\text{DepartmentID} \rightarrow \text{DepartmentName}$. Therefore StudentID indirectly determines DepartmentName through DepartmentID , creating a transitive dependency.

Anomalies

Poorly designed relations can suffer insertion, update and deletion anomalies. An insertion anomaly occurs when a fact cannot be recorded without adding an unrelated fact. An update anomaly occurs when the same fact is duplicated across multiple rows and all copies must be changed consistently. A deletion anomaly occurs when removing one fact accidentally removes another useful fact. Normalization decomposes relations to reduce these anomalies while preserving important dependencies and, ideally, lossless reconstruction.

Normalization — 1NF

First Normal Form requires attributes to contain atomic values under the relational interpretation and disallows repeating groups within a row. A column should represent a single logical value for each tuple. A table containing a phone-number list in one cell may violate the intended 1NF interpretation if the application treats those numbers as multiple values. The remedy depends on requirements: multiple values may be represented by separate rows in a related table.

Normalization — 2NF

Second Normal Form requires 1NF and that every non-prime attribute be fully functionally dependent on the whole candidate key, not merely part of a composite key. If $\text{Enrollment}(\text{StudentID}, \text{CourseID}, \text{StudentName}, \text{CourseName}, \text{Grade})$ has a composite key $(\text{StudentID}, \text{CourseID})$, StudentName depends only on StudentID and CourseName only on CourseID . Those partial dependencies indicate that the relation should be decomposed.

Normalization — 3NF

Third Normal Form requires 2NF and eliminates relevant transitive dependencies of non-key attributes on a key. A common formal condition is that for every nontrivial FD $X \rightarrow A$, X is a superkey or A is a prime attribute. The practical goal is to keep facts about one determinant in the appropriate relation. For example, if $\text{EmployeeID} \rightarrow \text{DepartmentID}$ and $\text{DepartmentID} \rightarrow \text{DepartmentName}$, DepartmentName should normally be stored with Department rather than repeated for every employee.

BCNF

Boyce-Codd Normal Form is stronger than 3NF. For every nontrivial functional dependency $X \rightarrow Y$, X must be a superkey. A relation can satisfy 3NF yet violate BCNF in certain dependency patterns. BCNF is useful when stronger elimination of redundancy is required, although decomposition can sometimes make dependency preservation more difficult.

4NF

Fourth Normal Form addresses multivalued dependencies. A relation can be free of problematic functional dependencies and still duplicate independent multi-valued facts. If a person independently has multiple skills and multiple hobbies, storing both sets in one relation creates unnecessary combinations. Decomposing the independent multivalued facts into separate relations can satisfy 4NF.

5NF

Fifth Normal Form, also called Project-Join Normal Form, addresses join dependencies. A relation is decomposed when facts can be represented more naturally by several projections and the original relation can be reconstructed through joins without introducing spurious tuples. 5NF is uncommon in ordinary application design but is important for understanding the theoretical limits of normalization.

Domain-Key Normal Form

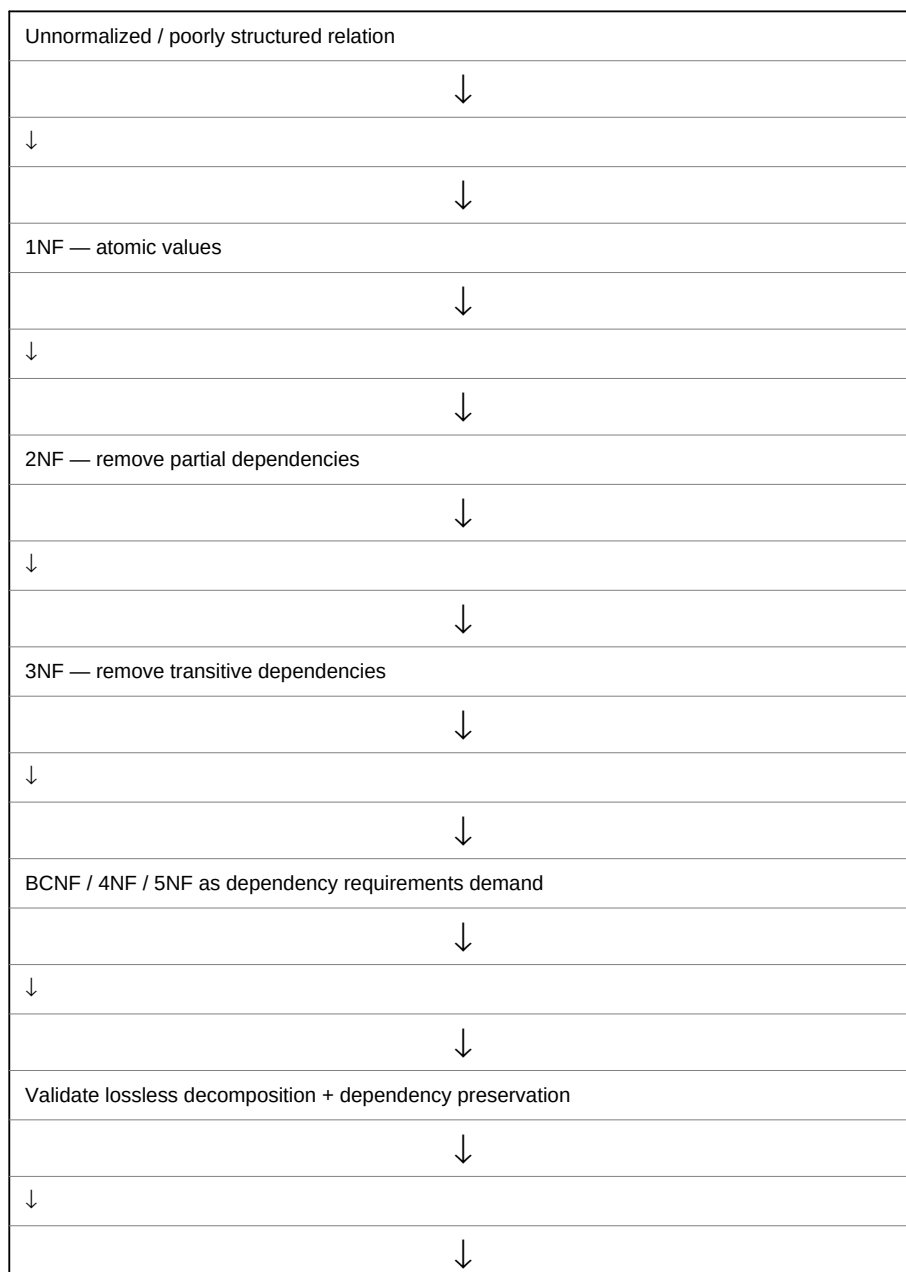
Domain-Key Normal Form (DKNF) aims to ensure that all constraints on a relation are consequences solely of domain constraints and key constraints. It is a very strong theoretical normal form. Many real systems do not reach DKNF because business rules often involve constraints that are not expressible solely through domains and keys.

Denormalization

Denormalization deliberately introduces controlled redundancy to improve read performance or simplify frequently executed queries. Examples include storing a derived value or duplicating a descriptive attribute to avoid repeated joins.

Denormalization can reduce query cost but increases storage, update complexity and risk of inconsistent copies. It should be a deliberate performance decision supported by workload measurements rather than a substitute for sound initial design.

Normalization decision flow



Denormalize only when justified by workload

PART III — SQL AND QUERY OPTIMIZATION

SQL Standards

SQL is a declarative language standardized by ISO/IEC and adopted in variants by database vendors. The standard defines language concepts, but commercial systems provide extensions and differ in syntax, data types, procedural features and implementation details. SQL is generally divided conceptually into definition, manipulation, control and transaction statements. SQL queries describe the desired result; the DBMS chooses an execution strategy.

SQL Data Types

Common SQL data types include integer and decimal numeric types, character types such as CHAR and VARCHAR, date/time types, Boolean-like types depending on implementation, binary types and large object types. Exact names and capabilities vary among products. Choosing an appropriate type is important for correctness, storage and index efficiency. Numeric precision should be selected according to the required range and exactness rather than treating every number as a generic floating-point value.

Database Objects

Common database objects include tables, views, indexes, constraints, schemas, sequences/identity generators, stored procedures, functions, triggers and sometimes materialized views. A table stores relational data. A view is a stored query definition that presents derived data. An index accelerates selected access paths. Constraints enforce rules such as uniqueness and referential integrity.

DDL — Data Definition Language

DDL defines or changes database structures. Typical commands include CREATE, ALTER and DROP. CREATE TABLE establishes columns and constraints; ALTER changes an existing object; DROP removes an object. DDL affects schema objects rather than simply changing individual row values. Exact transactional behaviour of DDL varies by DBMS.

DML — Data Manipulation Language

DML changes or retrieves data. INSERT adds rows, UPDATE modifies rows, DELETE removes rows and SELECT retrieves data. SELECT is often treated separately as query language, but it is commonly discussed with DML. A WHERE clause controls which rows are affected by UPDATE and DELETE. Omitting a restrictive WHERE condition can modify every row, so predicates are an essential part of safe data manipulation.

DCL — Data Control Language

DCL controls authorization. GRANT gives privileges, while REVOKE removes privileges. Permissions can apply to objects such as tables, views and routines, depending on the DBMS. Authorization separates the ability to access data from the mere existence of that data, supporting least-privilege security.

TCL — Transaction Control Language

Transaction-control statements include COMMIT and ROLLBACK, with SAVEPOINT commonly used to mark an intermediate point. COMMIT makes a transaction's changes durable according to the DBMS's transaction semantics. ROLLBACK reverses uncommitted changes. Transactions group logically related operations so the database can maintain consistency despite failures or concurrent users.

Example SQL:

```
CREATE TABLE Department (  
  DepartmentID INT PRIMARY KEY,  
  DepartmentName VARCHAR(100) NOT NULL UNIQUE  
);  
  
CREATE TABLE Employee (  
  EmployeeID INT PRIMARY KEY,  
  EmployeeName VARCHAR(100) NOT NULL,  
  DepartmentID INT,  
  FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)  
);
```

```
SELECT EmployeeName
FROM Employee
WHERE DepartmentID = 10;
```

Embedded SQL

Embedded SQL places SQL statements inside a host programming language such as C. The program can use host variables to pass values to SQL statements and receive results. A precompiler may translate embedded SQL into calls understood by the DBMS interface. The approach allows database operations to participate directly in application logic while retaining SQL as the data-access language.

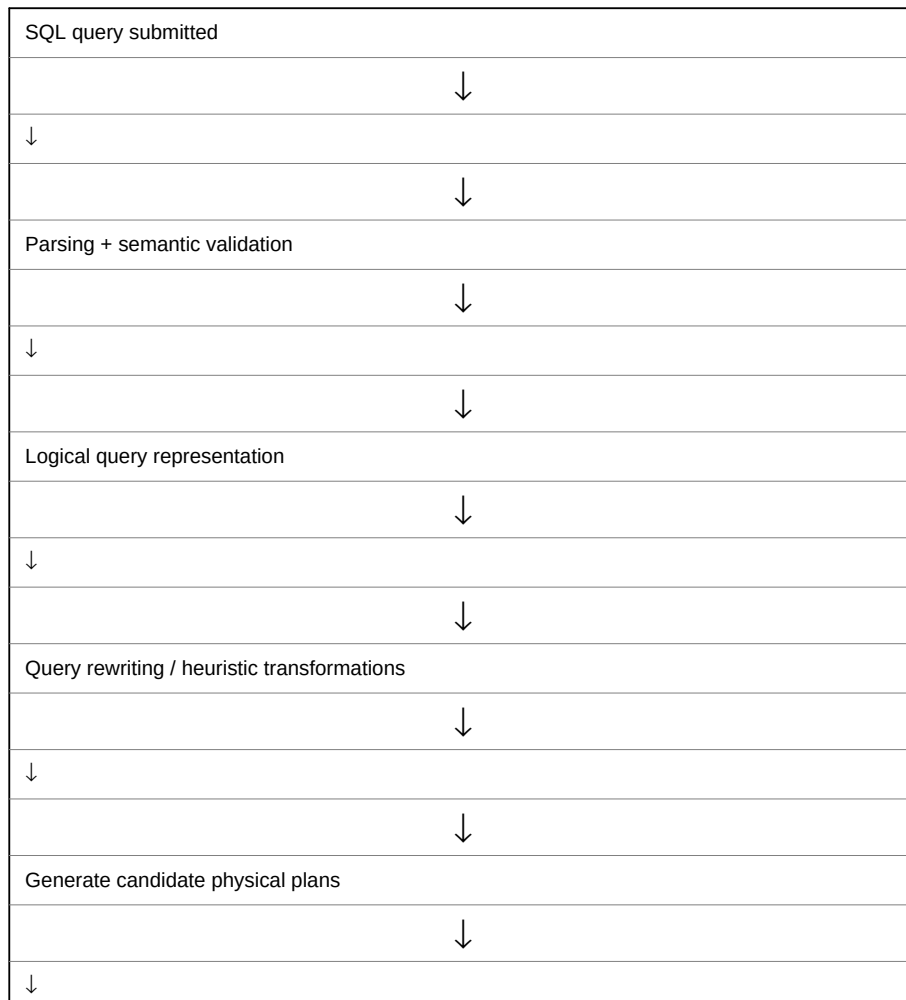
Static vs Dynamic SQL

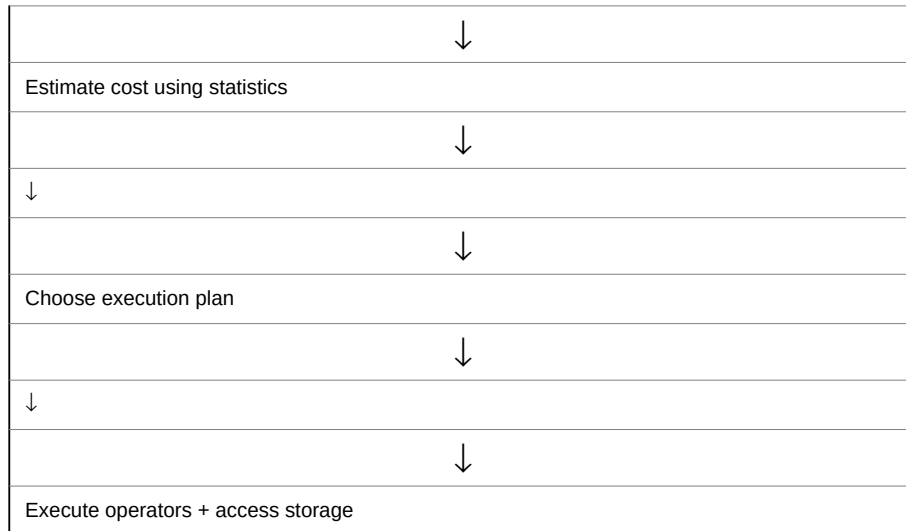
Static SQL has the statement structure determined before execution, often at compile or preparation time. Dynamic SQL constructs or supplies SQL statements at runtime, allowing more flexible queries. Static SQL can simplify validation and optimization when query structure is known. Dynamic SQL is useful when table names, predicates or other statement components genuinely need to vary, but careless construction can create SQL injection vulnerabilities. Parameterized statements are preferred for user-supplied values.

Query Processing

When a DBMS receives a query, it generally parses and validates the SQL, translates it into an internal representation, generates alternative execution strategies, estimates their costs and selects a plan. The chosen plan is then executed by operators such as scans, selections, projections, joins, sorting and aggregation. The optimizer is not merely trying to make SQL text shorter. It is selecting physical operations and access paths that are expected to minimize resource cost.

Query processing pipeline





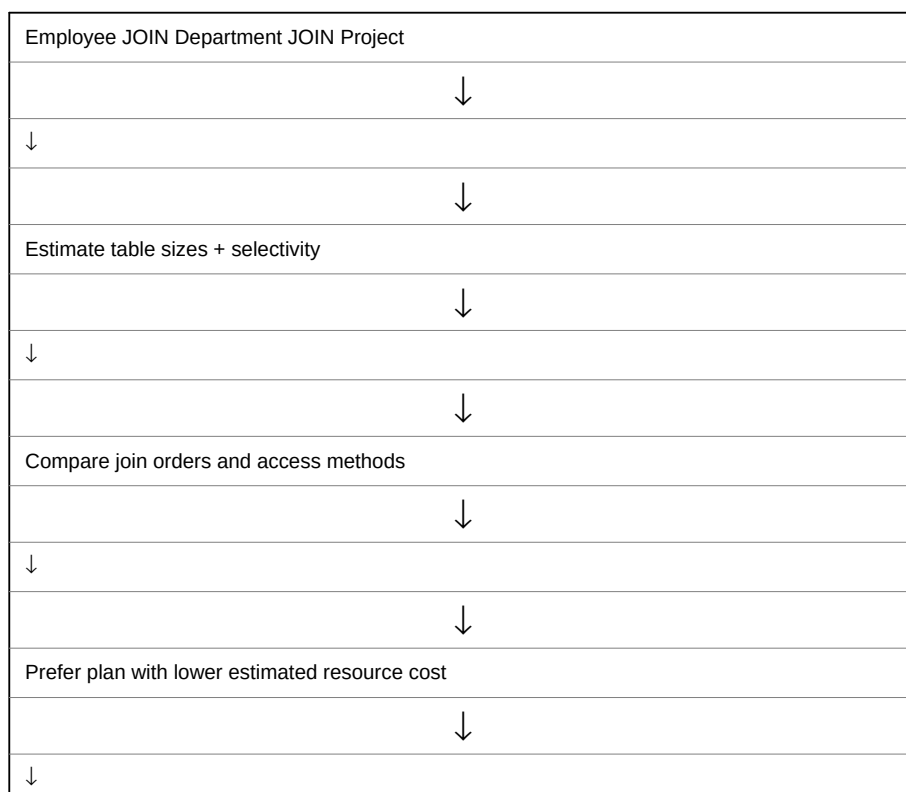
Heuristics in Query Optimization

Heuristic optimization applies generally useful transformations. Selection pushdown attempts to apply restrictive predicates early, reducing the number of rows carried into later operations. Projection pushdown removes unnecessary columns early. Reordering joins can place smaller or more selective intermediate results earlier. Heuristics are useful because exhaustive plan enumeration can be expensive. However, a heuristic that is usually beneficial is not universally optimal, so cost-based optimization and statistics remain important.

Cost Estimates in Query Optimization

A cost-based optimizer estimates the cost of alternative plans using statistics such as table cardinality, number of pages, value distributions and index information. Costs may represent disk I/O, CPU, memory, network transfer or a weighted combination. For joins, the optimizer estimates the size of intermediate results because an underestimated or overestimated cardinality can cause a poor choice of join order or access method. Accurate statistics are therefore essential for good plans.

Example of join-plan reasoning



Execute selected physical plan

PART IV — TRANSACTION PROCESSING AND CONCURRENCY CONTROL

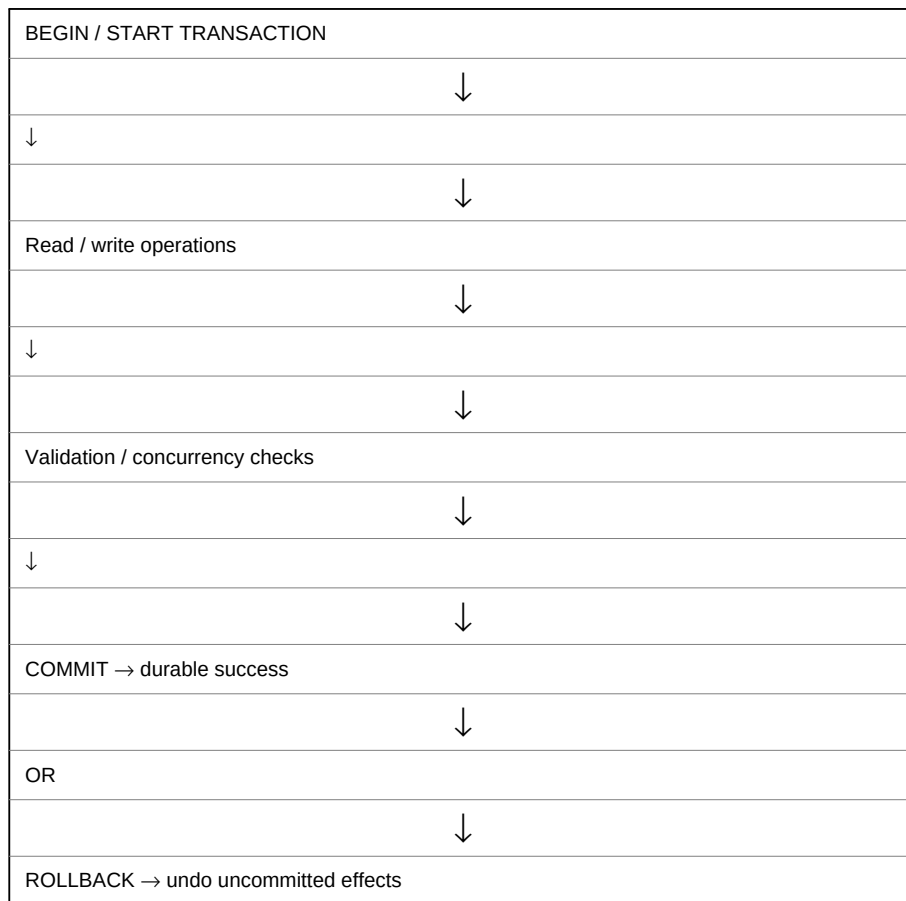
Introduction to Transactions

A transaction is a logical unit of database work consisting of one or more operations. A transfer between accounts, for example, may require debiting one account and crediting another. The transaction should not leave the database with only one side completed. The DBMS provides mechanisms to execute transactions reliably even when several users operate simultaneously or a system failure occurs.

Properties of a Transaction — ACID

Atomicity means the transaction's operations are treated as one unit: either all required effects occur or none do. Consistency means a committed transaction preserves defined database integrity constraints. Isolation means concurrent execution should behave according to the isolation guarantees of the DBMS. Durability means committed effects survive appropriate failures through logging and recovery mechanisms. ACID is a conceptual framework; the exact isolation level and concurrency behaviour depend on the DBMS.

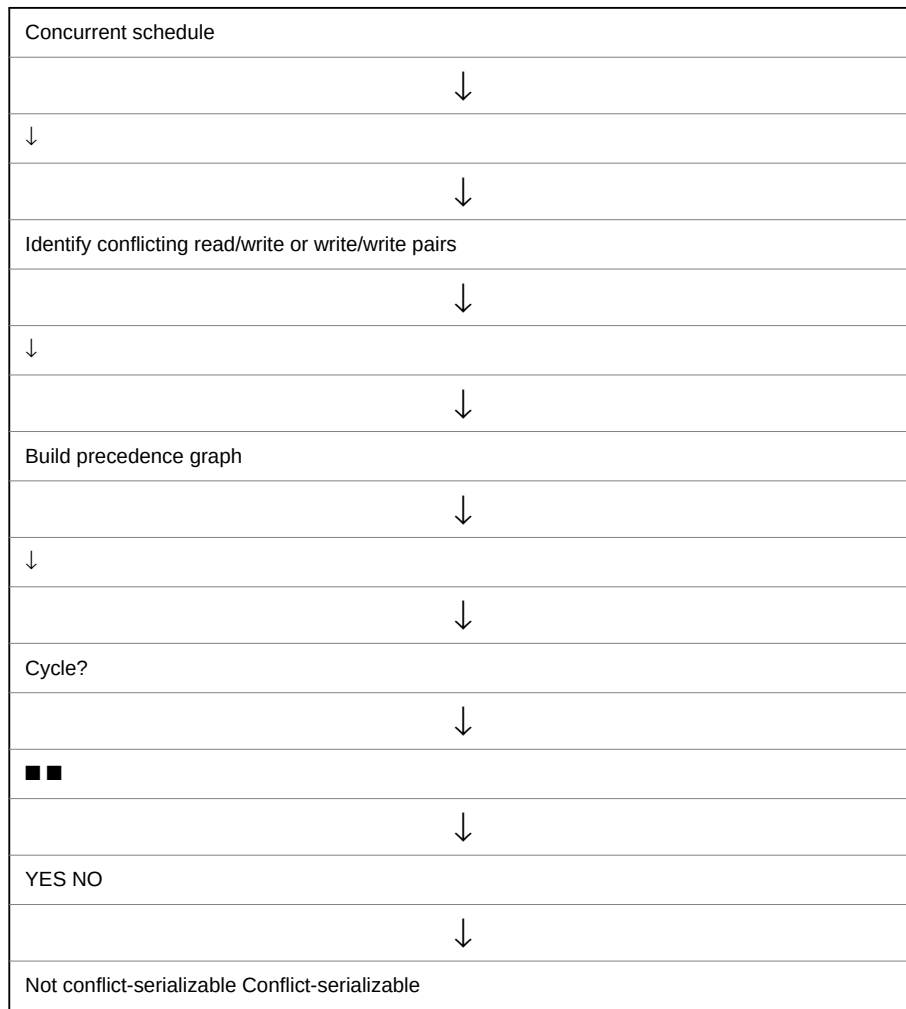
Transaction lifecycle



Serializability

A serial schedule executes transactions one after another with no interleaving. A concurrent schedule interleaves operations from multiple transactions. Serializability asks whether the concurrent result is equivalent to some serial ordering of the same transactions. Conflict serializability is commonly analyzed using a precedence graph. Each node represents a transaction. A directed edge $T_i \rightarrow T_j$ indicates that an operation of T_i conflicts with and occurs before a conflicting operation of T_j . A schedule is conflict-serializable if its precedence graph is acyclic.

Conflict-serializability test



Concurrency Control

Concurrency control coordinates simultaneous transactions so that isolation and consistency are preserved. Without control, problems can include lost updates, dirty reads, non-repeatable reads and inconsistent analyses. Common approaches include locking, timestamp ordering, optimistic validation and multiversion concurrency control. Lock-based methods are especially important in traditional DBMS theory.

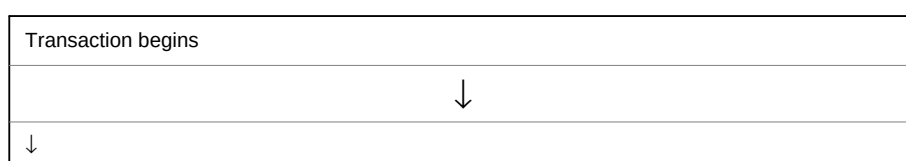
Locking Mechanisms

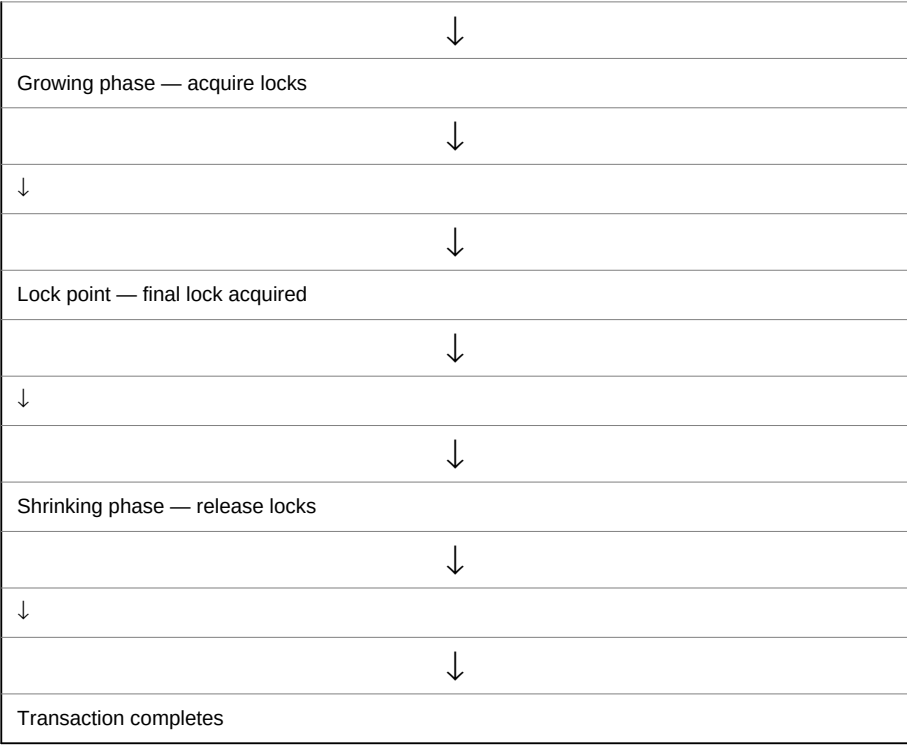
A shared lock (S) permits compatible reads, while an exclusive lock (X) is required for writing. Multiple transactions may normally hold shared locks on the same item, but an exclusive lock conflicts with other shared or exclusive locks. A lock manager tracks locks and determines whether requested locks are compatible. Correct lock protocols are necessary to prevent unsafe interleavings.

Two-Phase Locking (2PL)

Two-phase locking divides a transaction's locking behaviour into a growing phase and a shrinking phase. During the growing phase, locks can be acquired but not released. During the shrinking phase, locks can be released but not acquired. Basic 2PL guarantees conflict serializability. Strict 2PL retains exclusive locks until commit or rollback, which simplifies recovery and prevents other transactions from reading uncommitted writes. Rigorous variants may retain both shared and exclusive locks until completion.

Two-phase locking

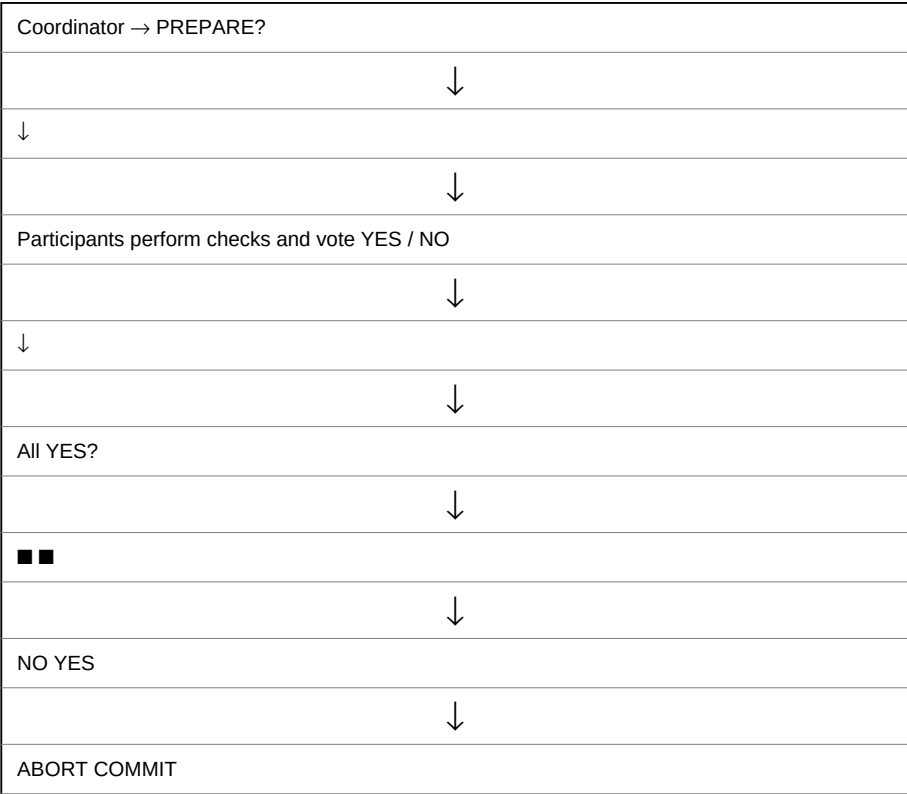


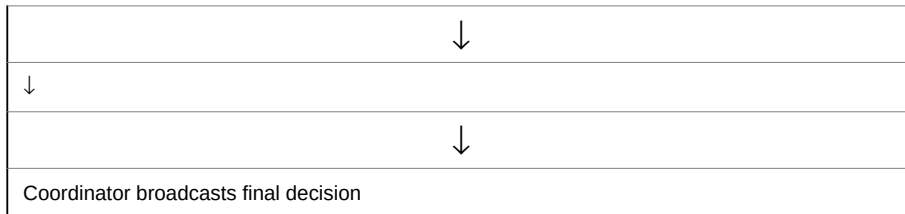


Two-Phase Commit Protocol (2PC)

Two-Phase Commit is a distributed transaction protocol used to reach a common commit/abort decision among participating sites. It has a coordinator and participants. In phase one, the coordinator asks participants to prepare. Each participant records enough information to commit if instructed and responds yes/ready or no. In phase two, if all participants are ready, the coordinator sends commit; otherwise it sends abort. Participants then apply the decision and acknowledge. 2PC provides atomic agreement but can block under certain coordinator or communication failures, which is one reason more advanced distributed transaction systems use additional mechanisms.

2PC flow

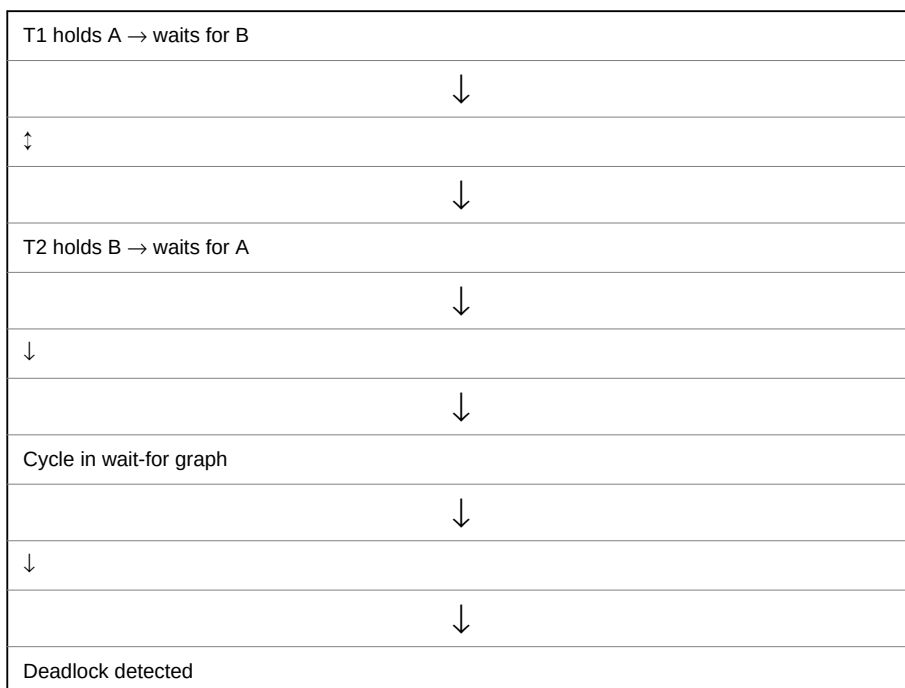




Deadlock

A deadlock occurs when transactions wait indefinitely for one another. A simple two-transaction cycle occurs when T1 holds a lock needed by T2 while T2 holds a lock needed by T1. Deadlocks can be handled through prevention, avoidance, detection and recovery. Detection commonly uses a wait-for graph: a cycle indicates deadlock. Recovery can abort one or more transactions, release their locks and restart them later. Prevention techniques impose ordering or priority rules that make cycles impossible or less likely.

Deadlock example



PART V — STORAGE, INDEXING, HASHING AND DATABASE TRENDS

RAID

RAID (Redundant Array of Independent Disks) combines multiple physical disks to improve performance, reliability or both. Different RAID levels distribute data and/or redundancy differently. RAID 0 stripes data without redundancy, providing performance but no fault tolerance. RAID 1 mirrors data. RAID 5 uses block-level striping with distributed parity and can tolerate one disk failure. RAID 6 uses dual parity and can tolerate two disk failures. RAID 10 combines mirroring and striping. RAID is a storage reliability/performance technology, not a substitute for database backup.

Simplified RAID ideas

RAID 0 → Striping → Performance, no redundancy
↓
RAID 1 → Mirroring → Redundancy
↓
RAID 5 → Striping + distributed parity → one-disk fault tolerance
↓
RAID 6 → Striping + dual parity → two-disk fault tolerance
↓
RAID 10 → Mirroring + striping → performance + redundancy

File Organization

Database files are organized into pages or blocks containing records. Common organizations include heap files, sequential/sorted files and hashed files. Heap organization stores records without a required ordering and makes arbitrary insertion simple. Sorted organization keeps records ordered by a search key and supports efficient ordered processing. Hash organization maps keys to storage buckets. The DBMS chooses a file organization based on workload, access patterns and update requirements.

Organization of Records in Files

Records may be fixed-length or variable-length. Fixed-length records simplify address calculation because the physical position can be derived from record number. Variable-length records require metadata such as offsets or slot directories to locate fields safely. A page may contain a page header, records and free space. Slotted-page organization is common because it supports variable-length records and allows records to move within a page while logical record references remain manageable.

Typical slotted-page idea

Page
↓
■■■■ Header / slot directory
↓
■■■■ Slot entries → record locations
↓
■■■■ Record data
↓

Indexing and Hashing

An index is an auxiliary structure that provides a faster access path to records based on one or more search-key attributes. Indexes may be ordered, tree-based or hash-based. Indexing is beneficial when queries frequently search, join or order by indexed attributes. However, every relevant INSERT, UPDATE or DELETE may require index maintenance, and indexes consume storage.

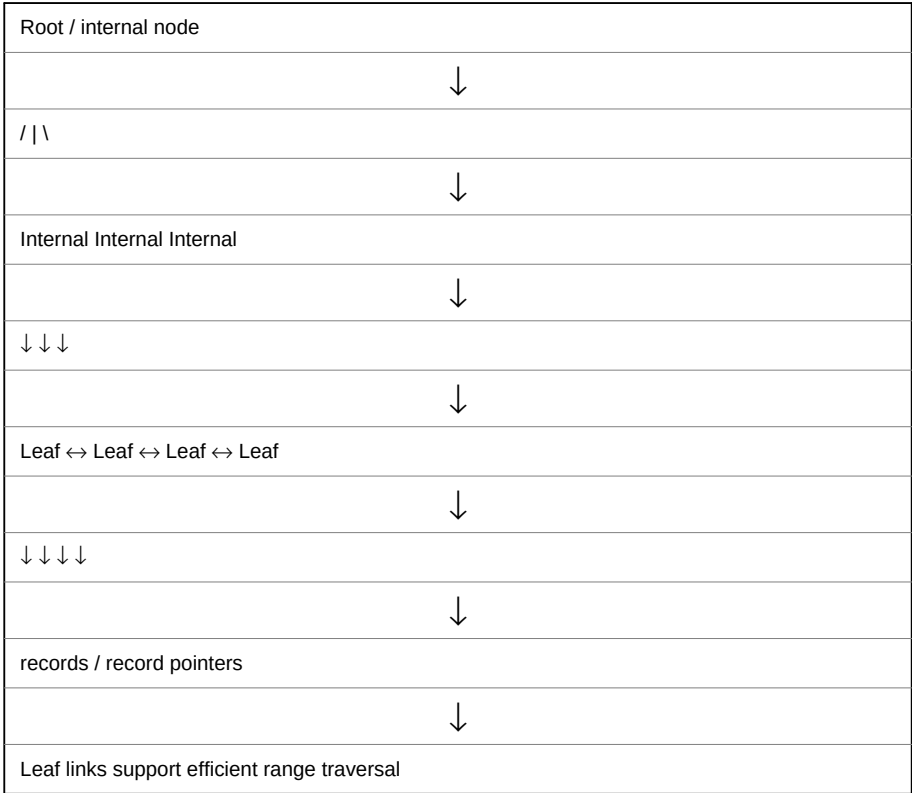
Ordered Indices

An ordered index stores search-key values in sorted order. A primary index may be associated with a file ordered on its primary key. A clustering index is associated with an ordering of records on a non-key attribute. A secondary index provides an additional access path. Dense indexes contain an index entry for every search-key value/record as defined by the design, while sparse indexes contain fewer entries and depend on an ordered data file. The distinction affects storage and lookup behaviour.

B+ Tree Index Files

A B+ tree is a balanced multiway search tree widely used for database indexing. Internal nodes contain separator keys and child pointers; leaf nodes contain search keys and record pointers or record references. Leaves are linked sequentially, making range scans efficient. All actual search entries are maintained at the leaf level in the standard B+ tree organization. The tree remains balanced because splits and merges redistribute keys when nodes become too full or too empty. The high fan-out of B+ trees keeps height small, reducing the number of disk/page accesses required for indexed retrieval.

B+ tree structure



B Tree Index Files

A B-tree is also a balanced multiway search tree, but unlike a standard B+ tree, records or record pointers may be stored in both internal and leaf nodes. This means a successful search can terminate at an internal node. B-trees and B+ trees maintain logarithmic height. B+ trees are generally preferred in database systems because their high fan-out and linked leaves make range queries and sequential index scans particularly efficient.

B+ Tree vs B Tree

Feature	B-tree	B+ tree
Record references	May occur in internal and leaf nodes	Typically at leaf level
Leaf linkage	Not fundamental	Linked leaves are fundamental to range scans
Range queries	Good	Very efficient
Fan-out	High	Often higher because internal nodes store only separators/pointers

Static Hashing

In static hashing, the number of buckets is fixed or changes only through an explicit rebuilding process. A hash function maps search keys to buckets. Good hashing distributes records evenly. Collision resolution can use overflow buckets or chaining. Static hashing is effective when the expected data size is relatively stable, but performance can degrade as the table grows because buckets become crowded.

Dynamic Hashing

Dynamic hashing allows the number of buckets or the effective address space to grow as data grows. Techniques include extendible hashing and linear hashing. Extendible hashing uses a directory and bucket depths, while linear hashing expands buckets incrementally according to a controlled split sequence. Dynamic hashing reduces the need for full-table rebuilding when the data volume changes significantly.

Static vs dynamic hashing

Hash key
↓
↓
↓
Static hashing → fixed bucket structure → overflow as data grows
↓
OR
↓
Dynamic hashing → bucket splits / directory growth → adapt to data size

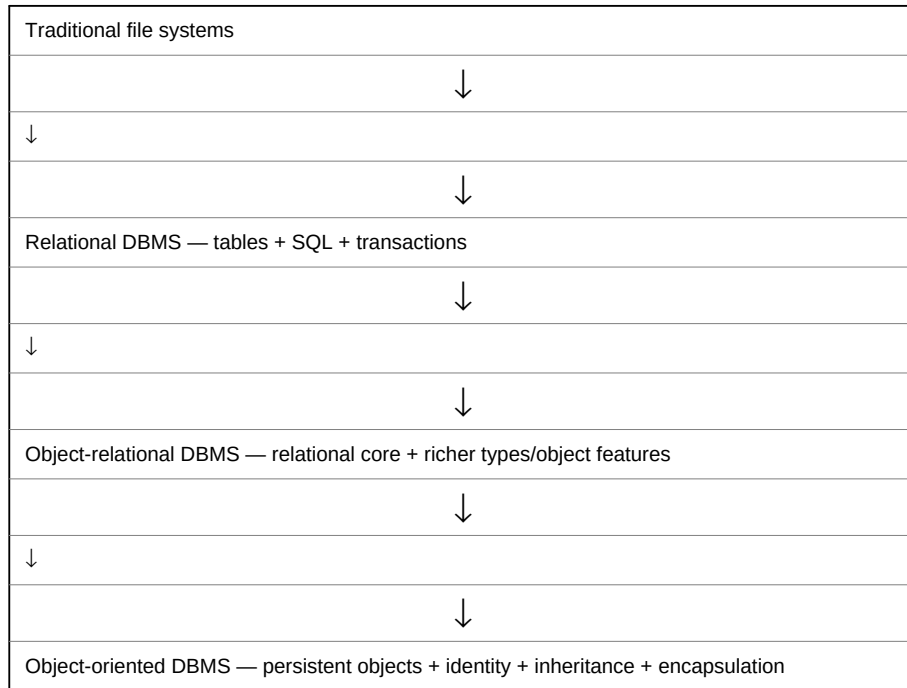
Object-Oriented Database Management Systems (OODBMS)

An object-oriented DBMS stores persistent objects and supports object identity, classes, inheritance, encapsulation and complex object structures. Instead of mapping every complex object into a set of flat rows, an OODBMS can preserve object structure more directly. Object identity means an object can have an identity independent of its attribute values. Inheritance allows specialized classes to reuse and extend general classes. Encapsulation groups state and behaviour around objects. OODBMSs can be useful for engineering, multimedia, scientific and other applications involving complex interconnected objects.

Object-Oriented Relational Database Management Systems (ORDBMS)

An object-relational DBMS extends the relational model with selected object-oriented capabilities while retaining tables and SQL as central concepts. Features may include user-defined types, structured types, arrays or collections, references and methods, depending on the implementation. ORDBMSs aim to combine the mature relational ecosystem—SQL, transactions, constraints and relational query processing—with richer data types and object-like modelling.

Database technology spectrum



PART VI — CONSOLIDATED CONCEPT MAP

Unit IV concept map

DBMS fundamentals
↓
↓
↓
File organization + relational model + relational algebra
↓
↓
↓
ER / EER conceptual design
↓
↓
↓
Functional dependencies + normalization
↓
↓
↓
SQL — DDL / DML / DCL / TCL + embedded/dynamic SQL
↓
↓
↓
Query processing + optimization
↓
↓
↓
Transactions + ACID + serializability + locking
↓
↓
↓
2PL + 2PC + deadlock handling
↓
↓
↓
Storage — RAID + files + indexes + hashing
↓

↓	
	↓
B-tree / B+ tree + static/dynamic hashing	
	↓
↓	
	↓
OODBMS + ORDBMS	

KEY COMPARISONS

File system vs DBMS	Application-managed files; limited sharing/integrity/recovery	Centralized data management; queries, constraints, security, concurrency and recovery
Selection vs Projection	Selection chooses rows based on a predicate	Projection chooses attributes/columns
2NF vs 3NF	Removes partial dependency on part of a composite key	Removes relevant transitive dependency
B-tree vs B+ tree	Records may be represented in internal and leaf nodes	Record references normally concentrated at leaves; linked leaves aid range scans
Static vs Dynamic hashing	Bucket structure essentially fixed	Bucket structure adapts as data grows
OODBMS vs ORDBMS	Object model is central	Relational model remains central with object extensions
Static SQL vs Dynamic SQL	Statement structure known ahead of execution	Statement can be constructed/prepared at runtime
2PL vs 2PC	Concurrency-control locking protocol	Distributed commit agreement protocol